3. METHODOLOGY

3.1 Adam-NV Overview

Standard first-order adaptive optimizers such as Adam maintain a fixed exponential decay rate β_1 governing the contribution of historical gradient information to the first-moment estimate. This fixed rate implicitly assumes that the reliability of the instantaneous gradient signal is constant throughout training, an assumption that does not hold when gradients are subject to varying levels of stochastic corruption, whether from minibatch sampling, data corruption, or other sources of training-time noise.

Adam-NV (Noise-aware Variance-adaptive momentum) relaxes this assumption by allowing β_1 to vary per parameter and per step, as a function of a locally estimated noise-to-signal ratio σ^2. The estimator σ^2 is computed from the optimizer's own first- and second-moment statistics at the previous step, avoiding circular dependence between the noise estimate and the gradient it is derived from. A warm-up rectification term suppresses the influence of this estimate during early training, when moment estimates are not yet reliable. The resulting controller lowers β_1 — reducing reliance on historical momentum in favor of the instantaneous gradient — when σ^2 is high, and raises β_1 toward its nominal value when σ^2 is low. Full derivations and bias-correction details are provided in the Theory section; this summary is included only to motivate the experimental design that follows.

3.2 Experimental Philosophy

The evaluation protocol deliberately separates two questions that are often conflated in optimizer evaluation studies: whether a proposed mechanism behaves as designed (mechanism validation), and whether that behavior yields measurable improvement in optimization outcomes (performance evaluation). Conflating these questions risks two failure modes: attributing performance differences to a mechanism that was never actually verified to be active, or dismissing a mechanism as ineffective when it was, in fact, never given the opportunity to engage under the tested conditions.

Each experiment in this study is therefore designed to isolate a single controlled variable while holding all other factors — architecture, optimizer hyperparameters other than the one under test, data pipeline, and random seed protocol — fixed across compared conditions. This framing treats each experiment as a targeted causal probe rather than as a leaderboard-style comparison, and each is stated in terms of an explicit hypothesis, a manipulated variable, and a set of metrics chosen to test that hypothesis specifically. Consequently, experiments are interpreted hierarchically: mechanism-level validation precedes, and is treated as a precondition for, any claim regarding optimization performance.

3.3 Experimental Environment

All experiments were conducted using PyTorch 2.11.0 with torchvision 0.26.0 on Python 3.12.13. Training was performed on an NVIDIA Tesla T4 GPU provided through Google Colab, using CUDA 12.8 and cuDNN 9.19. Unless otherwise specified, cuDNN benchmark and deterministic settings were left at their PyTorch defaults; a --deterministic flag is available to force deterministic convolution algorithm selection, but was not used for the primary reported experiments, to avoid introducing a settings mismatch relative to the baseline convergence experiments (Section 3.9, E1), which predate the introduction of this flag.

3.4 Datasets

The primary evaluation dataset is CIFAR-10, comprising 60,000 32×32 color images across 10 classes, with the standard 50,000/10,000 train/test split. Training images were augmented with random 32×32 crops (4-pixel padding) and random horizontal flips, followed by per-channel normalization using dataset statistics (per-channel mean (0.4914, 0.4822, 0.4465), standard deviation (0.2470, 0.2435, 0.2616)). Test images were normalized identically, without augmentation. Throughout this paper, "validation accuracy" and "validation loss" refer to metrics computed on this held-out 10,000-image test split; no separate validation split was carved out of the training data. As no architecture search, hyperparameter tuning, or model selection was performed using this split (Section 3.5), its use for reporting does not constitute test-set leakage in the conventional sense; it is used exclusively as a fixed, held-out evaluation set.

Two forms of controlled data corruption were introduced for specific experiments, in addition to CIFAR-10 in its unmodified form. Symmetric label noise (Section 3.9, E11) was generated by selecting a fixed fraction of training samples uniformly at random and reassigning each to a different class label, drawn uniformly from the remaining nine classes. This corruption was generated once, prior to training, using a dedicated random seed independent of all other stochastic elements of the training pipeline (Section 3.8), and was applied exclusively to the training split; the test split was never modified. Synthetic Gaussian perturbation of gradients (Section 3.9, E2; Section 3.7) is a training-time procedure rather than a dataset modification and is described separately below.

3.5 Model Architecture

All experiments use a CIFAR-adapted ResNet-18, obtained by replacing the standard ImageNet-oriented stem (a 7×7 stride-2 convolution followed by a max-pooling layer) with a 3×3 stride-1 convolution and an identity operation in place of the max-pooling layer, consistent with common practice for CIFAR-scale ResNet variants. This architecture was held identical across every optimizer, noise condition, and hyperparameter setting tested in this study; no architecture search or per-condition tuning was performed. For a given random seed, model initialization was performed identically regardless of which optimizer or noise condition was subsequently applied, ensuring that any observed differences between conditions cannot be attributed to differences in starting parameters. Holding architecture and initialization constant isolates the optimizer configuration and injected

perturbation as the sole manipulated variables in each comparison.

3.6 Optimizers

The primary comparison throughout this study is between standard Adam (β_1=0.9, β_2=0.999, ε=1×10⁻⁸) and Adam-NV. Wherever the two are compared directly, all hyperparameters other than the adaptive β_1 mechanism itself — learning rate, β_2, ε, and weight decay — were held identical between the two optimizers, so that any behavioral or performance difference can be attributed specifically to the presence or absence of adaptive momentum control. Learning rate followed a cosine annealing schedule with no warm-restarts, applied identically regardless of optimizer.

Two additional established optimizers, AdamW and AdaBelief, were included as baselines exclusively in the convergence-benchmarking experiment (Section 3.9, E1), using their respective literature-default hyperparameter configurations. These baselines were not carried forward into subsequent experiments, which focus specifically on isolating the adaptive-β_1 mechanism relative to standard Adam.

Table 1 reports the core hyperparameters and training configuration held fixed across experiments unless explicitly varied as the manipulated variable of a given experiment (for example, batch size in E12, or γ in E7-C).

Table 1. Core training configuration.

Parameter	**Value**
Learning rate (initial)	1×10⁻³
LR schedule	Cosine annealing, no warm restarts, T_max = total epochs
Epoch budget	60
Default batch size	128 (varied in E12: {32, 128, 256})
β_2 (Adam, Adam-NV)	0.999
ε (Adam, Adam-NV)	1×10⁻⁸
Weight decay (Adam, Adam-NV)	0.0
Adam-NV: β_base (default)	0.9
Adam-NV: β_min	0.1
Adam-NV: γ (default)	1.0 (varied in E7-C: {0, 0.5, 1, 2})
Adam-NV: α (ρ smoothing)	0.95
Adam-NV: k (warm-up constant)	100
Adam-NV: λ (denominator stabilizer)	0.1
AdamW: weight decay	1×10⁻⁴
AdaBelief: rectify / decoupled decay	disabled (rectify=False, weight_decouple=False)

3.7 Noise Injection

Controlled gradient corruption is introduced, where specified, by the transformation

$$g_{noisy} = g + \sigma \cdot \|g\|_F \cdot \hat{\varepsilon}, \hat{\varepsilon} = \varepsilon / \|\varepsilon\|_F, \varepsilon \sim N(0, I)$$

applied independently to each parameter tensor, where $\|g\|_F$ denotes the Frobenius norm of that tensor's gradient. The perturbation direction ε is drawn from a standard multivariate Gaussian and subsequently normalized to unit Frobenius norm prior to scaling, which ensures that the injected perturbation magnitude is exactly $\sigma \cdot \|g\|_F$ regardless of the number of elements in the tensor. Without this normalization, the expected norm of an unnormalized Gaussian draw scales with the square root of tensor size, which would cause the effective noise-to-gradient ratio to vary systematically across parameter tensors of different shapes rather than being controlled by σ alone. This scaling behavior was verified through a dedicated calibration procedure prior to use in any reported experiment.

Noise injection is performed after the backward pass and before the optimizer step, using an in-place modification of the gradient tensor, and is applied identically to every optimizer under comparison at a given noise level, ensuring that compared optimizers receive byte-identical perturbations at each step.

Random draws used for noise injection are sourced from a dedicated, explicitly seeded pseudorandom generator, kept independent from the generator controlling DataLoader minibatch shuffling, to prevent interactions between perturbation generation and minibatch ordering across noise-level conditions (Section 3.8).

3.8 Reproducibility Protocol

Each training run is controlled by a single integer seed, from which all stochastic elements of that run are deterministically derived, using three independent pseudorandom generator instances rather than a single shared stream. A first generator, seeded directly from the run seed, controls model weight initialization and any operations drawing from the global PyTorch random state. A second, independently seeded generator controls DataLoader minibatch shuffling. A third, independently seeded generator controls gradient-noise injection (Section 3.7) where applicable. Where label noise is used (Section 3.9, E11), a fourth independently seeded generator controls the selection and reassignment of corrupted training labels. This separation ensures that enabling or varying any one stochastic process (for example, injected gradient noise or label corruption) cannot alter the realized sequence of any other stochastic process across compared conditions.

Unless a run is explicitly configured for deterministic execution, cuDNN convolution algorithm selection is left at PyTorch's default (non-forced) behavior, consistent across all conditions within a given experiment.

All runs log per-epoch training loss, training accuracy, validation loss, validation accuracy, learning rate, and wall-clock epoch duration to CSV. For Adam-NV, per-epoch logs additionally include the mean, minimum, and maximum of β_1 across parameter tensors, together with mean σ^2 and mean ρ (the exponentially smoothed noise signal from which β_1 is derived;

see Section 2 [AUTHOR TO COMPLETE: confirm Theory section number]). An optional higher-resolution logging mode records these same mechanism statistics, together with pre- and post-injection gradient norms, at a configurable per-step interval, used to verify that injected noise reaches the optimizer at the intended magnitude and to examine within-epoch dynamics at finer resolution than per-epoch logging permits.

3.9 Experimental Design

E1 — Baseline Convergence Benchmarking

Goal. To establish that Adam-NV's behavior on unperturbed data is consistent with standard first-order adaptive optimizers, prior to evaluating its behavior under controlled perturbation.

Design. Adam, AdamW, AdaBelief, and Adam-NV were each trained under identical architecture, data pipeline, and epoch budget, with no injected gradient or label noise, using literature-default hyperparameters for each optimizer.

Metrics. Final validation accuracy, epochs required to reach a fixed validation-accuracy threshold, and validation-loss area-under-curve across training.

E2 (H2a) — Mechanism Validation Under Gradient Noise

Goal. To test whether Adam-NV's adaptive controller responds to controlled, injected gradient noise in the direction predicted by its design.

Hypothesis. As injected noise level σ increases, mean β_1 is expected to decrease and mean σ^2 is expected to increase, reflecting reduced trust in the instantaneous gradient signal under higher corruption.

Design. Adam-NV was trained under four noise levels, $\sigma \in \{0.0, 0.05, 0.5, 2.0\}$, injected as described in Section 3.7, with all other configuration held fixed across noise levels. These levels were selected to span a wide range of perturbation magnitude, from sub-threshold to substantially larger than typical gradient norms, informed by preliminary calibration of injected-noise magnitude relative to observed gradient norms on this architecture (Section 3.7).

Metrics. Per-epoch and per-step mean β_1 and mean σ^2, tracked across the full noise-level sweep.

E2 (H2b) — Performance Robustness Under Gradient Noise

Goal. To test whether Adam-NV's final accuracy is less degraded by injected gradient noise than that of standard Adam, under matched noise conditions.

Design. Adam and Adam-NV were each trained under the same four noise levels used in E2 (H2a), with byte-identical injected perturbations at each step (Section 3.7).

Metric. Δacc, defined per optimizer as validation accuracy under the clean (σ=0) condition minus validation accuracy under a given noisy condition, compared between optimizers at matched σ.

E7-A — Adaptive versus Fixed β_1 Ablation

Goal. To determine whether any observed behavior of Adam-NV can be attributed to adaptive (time-varying) β_1, as distinct from Adam-NV converging on an effective constant value.

Design. Setting Adam-NV's adaptation-strength parameter γ to zero causes β_1 to remain fixed at the configured constant β_base for the duration of training, while preserving exact bias correction, since the running product $P_t = \beta_base^t$ under this configuration matches the closed-form expression used in standard Adam. This provides a fixed-β_1 variant of Adam-NV without requiring a separate optimizer implementation. The fixed value used for β_base in this comparison was set to the empirical, trailing-epoch-averaged value that full adaptive Adam-NV converges to under the matched noise condition, so that the comparison isolates the effect of time-varying adaptation itself, rather than confounding it with the choice of constant.

Metrics. Final validation accuracy, epochs to fixed accuracy thresholds, and cross-seed standard deviation of validation accuracy, compared between the adaptive and fixed-β_1 configurations.

E7-C — Adaptation-Strength (γ) Sweep

Goal. To characterize how Adam-NV's behavior and performance vary with adaptation strength γ, rather than to identify an optimal value of γ.

Design. Adam-NV was trained with $\gamma \in \{0, 0.5, 1, 2\}$, all other hyperparameters held fixed, under a gradient-noise condition selected as sufficient to elicit measurable mechanism activation (σ=2.0; see Section 3.9, E2).

Metrics. Mean β_1 and mean σ^2 trajectories across γ, together with final validation accuracy, epochs to fixed accuracy thresholds, and cross-seed standard deviation.

E9 — Computational Overhead

Goal. To quantify the additional per-step computational cost of Adam-NV's adaptive β_1 and σ^2 computation relative to standard Adam, since practical adoption of an optimizer depends on runtime cost in addition to optimization behavior.

Design. Wall-clock epoch duration was recorded for every run in this study using identical hardware and software configuration within each comparison. Because any fixed per-step computational overhead is expected to contribute a larger proportion of total epoch time when more optimizer steps occur per epoch, overhead was examined specifically across three batch sizes, {32, 128, 256}, allowing it to be characterized as a function of batch size rather than assumed constant. These are the same batch-size conditions used to probe mechanism-level batch-size sensitivity in E12.

Metric. Mean wall-clock time per training epoch, compared between Adam and Adam-NV within each matched configuration.

E11 — Label-Noise Evaluation

Goal. To evaluate whether Adam-NV's noise-detection mechanism responds to a structurally different form of training corruption than the zero-mean Gaussian gradient noise used in E2.

Design. Symmetric label noise (Section 3.4) was applied at two levels, 0% (clean) and 20% corrupted, with no injected gradient noise ($\sigma=0$), isolating label corruption as the sole perturbation. Because label corruption introduces a structured, biased perturbation rather than additive noise — a subset of training samples consistently pulls the model's predictions toward incorrect targets, rather than randomly perturbing the gradient at each step — E11 evaluates whether the adaptive controller responds differently to this form of corruption, and whether any such response depends on training progress. The full training schedule was therefore evaluated, rather than restricting analysis to early epochs.

Metrics. Mean β_1 and mean σ^2 trajectories across the full training schedule, together with final validation accuracy and its degradation relative to the clean condition, compared between Adam and Adam-NV.

E12 — Batch-Size Sensitivity

Goal. To test whether Adam-NV's controller responds to gradient noise that arises intrinsically from minibatch sampling, as distinct from noise introduced through explicit injection (E2).

Design. Adam-NV and Adam were each trained at three batch sizes, {32, 128, 256}, with no injected gradient or label noise. No other aspect of training configuration was altered between batch-size conditions. Because batch size and per-epoch optimizer step count are inversely related at a fixed epoch budget, a manipulated change in batch size necessarily co-varies with total step count. This coupling is an inherent property of fixed-epoch training and should therefore be considered when interpreting batch-size effects.

Metrics. Mean β_1 and mean σ^2 as a function of batch size, final validation accuracy and its difference between optimizers at each batch size, and per-epoch wall-clock time as a function of batch size (Section 3.9, E9).

3.10 Evaluation Metrics

Final validation accuracy and Δacc (Section 3.9, E2) serve as the primary outcome metrics throughout this study, as they most directly reflect practical optimization performance. Both are computed as a trailing five-epoch average rather than a single final-epoch value, reducing sensitivity to per-epoch measurement noise inherent to stochastic minibatch training.

Mean β_1 and mean σ^2 serve as mechanism-level metrics, quantifying whether and how strongly the adaptive controller responds to a given experimental manipulation. Following the optimizer's own diagnostic implementation, mean β_1 is computed as an unweighted average, across parameter tensors, of each tensor's own per-element mean β_1; this average is therefore not weighted by parameter count, a property retained for consistency with the diagnostic tooling used throughout this study and noted here for reproducibility.

Optimization-level metrics include epochs required to reach fixed validation-accuracy thresholds, used as a convergence-speed indicator; cross-seed standard deviation of validation accuracy, used as a stability indicator; and per-epoch wall-clock time, used as a runtime-overhead indicator (Section 3.9, E9).

3.11 Statistical Methodology

Each experimental condition was evaluated using multiple independent random seeds, typically four, with select comparisons extended to eight seeds where additional resolving power was warranted. Metrics subject to epoch-to-epoch noise, including β_1, σ^2, and validation accuracy, are reported as trailing five-epoch averages prior to any cross-condition comparison or statistical test.

Pairwise comparisons between conditions were conducted using Welch's unequal-variances t-test, selected over the standard two-sample t-test to avoid assuming equal variance across compared conditions, which is not guaranteed a priori. Effect sizes are additionally reported using Cohen's d, alongside p-values, in recognition that statistical significance and practical effect magnitude are distinct considerations. Primary statistical comparisons were defined prior to inspection of results wherever possible; comparisons selected after inspecting intermediate results are explicitly identified as exploratory rather than confirmatory. No family-wise multiple-comparisons correction (e.g., Holm-Bonferroni) was applied across experiments, as each pre-specified confirmatory comparison tests a distinct, non-overlapping hypothesis (for example, adaptive versus fixed β_1 in E7-A, or clean versus label-corrupted training in E11) rather than repeated tests of a single hypothesis; exploratory comparisons are reported descriptively and are not treated as evidence in their own right.

Prior to interpreting any observed difference between conditions as evidence for or against a hypothesis, a minimum-detectable-effect analysis was conducted, using the pooled within-condition standard deviation together with standard two-sample power formulas at $\alpha=0.05$ and 80% power. This analysis was used specifically to distinguish a genuine null result from an underpowered comparison, and informed decisions regarding whether additional seeds were warranted for a given comparison before that comparison was treated as conclusive.